UTS

PEMBELAJARAN MESIN



NAMA : Nuruliah Rifky

NIM : 202431075

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 10

ASISTEN : 1. Muhammad Caisar Gemilang Pratama

2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI ..... **ii**

BAB I PENDAHULUAN **1**

1.1 Rumusan Masalah 1

1.2 Tujuan Masalah 1

1.3 Manfaat Masalah 1

BAB II LANDASAN TEORI **2**

2.1 Decision Tree 2

2.1.1 Algoritma Decision Tree CART 2

2.2 Naive Bayes 3

BAB III HASIL **5**

3.1 Decision Tree – Naive Bayes 5

BAB IV PENUTUP **17**

DAFTAR PUSTAKA **18**

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Berdasarkan hasil praktikum UTS Pembelajaran Mesin yang telah dilakukan, terdapat beberapa rumusan masalah yang menjadi fokus pembahasan pada praktikum ini, yaitu:

1. Bagaimana konsep dasar dan cara kerja algoritma Decision Tree dan Naive Bayes dalam proses klasifikasi data?
2. Bagaimana penerapan algoritma Decision Tree dan Naive Bayes dalam mengolah dan mengklasifikasikan data?
3. Bagaimana perbedaan proses klasifikasi antara algoritma Decision Tree dan Naive Bayes?
4. Bagaimana hasil klasifikasi yang diperoleh menggunakan algoritma Decision Tree dan Naive Bayes?

1.2 Tujuan Masalah

Tujuan dari praktikum UTS Pembelajaran Mesin yang telah dilakukan adalah:

1. Memahami konsep dasar dan cara kerja algoritma Decision Tree dan Naive Bayes dalam proses klasifikasi data.
2. Menerapkan algoritma Decision Tree dan Naive Bayes dalam pengolahan dan klasifikasi data.
3. Mengetahui perbedaan proses klasifikasi antara algoritma Decision Tree dan Naive Bayes.
4. Menganalisis hasil klasifikasi yang diperoleh menggunakan algoritma Decision Tree dan Naive Bayes.

1.3 Manfaat Masalah

Manfaat yang diperoleh dari hasil praktikum UTS Pembelajaran Mesin ini adalah:

1. Menambah pengetahuan dan pemahaman mengenai konsep serta prinsip kerja algoritma Decision Tree dan Naive Bayes dalam pembelajaran mesin.
2. Meningkatkan keterampilan dalam menerapkan algoritma Decision Tree dan Naive Bayes untuk melakukan klasifikasi data.
3. Memberikan pemahaman mengenai perbedaan pendekatan klasifikasi berbasis pohon keputusan dan probabilitas.
4. Menjadi bahan pembelajaran dan referensi dalam memahami penggunaan algoritma klasifikasi untuk pengolahan serta analisis data.

BAB II

LANDASAN TEORI

2.1 Decision Tree

Decision Tree merupakan salah satu algoritma pada pengolahan data metode klasifikasi. Algoritma machine learning yang satu ini mempresentasikan strukturnya seperti struktur pohon untuk menemukan hasil keputusan. Konsep dari Decision Tree ini adalah dengan menyajikan pernyataan-pernyataan bersyarat pada setiap langkah yang bercabang untuk pengambilan keputusan berdasarkan perhitungan dari data set itu sendiri. Dalam metode Klasifikasi, algoritma Decision Tree merupakan yang paling banyak digunakan.

Decision Tree adalah salah satu metode klasifikasi yang memiliki bentuk menyerupai struktur pohon. Algoritma ini berfungsi untuk membantu dalam proses pengambilan keputusan dengan menyajikan alur logika yang divisualisasikan dalam bentuk cabang dan daun pada pohon keputusan. Model Decision Tree terdiri dari node dan cabang, di mana setiap node menggambarkan fitur atau atribut dari kategori yang akan diklasifikasikan, sedangkan setiap cabang menunjukkan nilai yang dimiliki oleh node tersebut.

Untuk mengkategorikan sekumpulan data tertentu, setiap atribut harus diterapkan melalui rentang node yang ada di node keputusan, dan setelah atribut mencapai node daun, data akan diklasifikasikan sesuai dengan kelas yang ada. Pada pohon keputusan ada tiga jenis node yang berbeda yaitu:

1. Root Node adalah node yang secara fisik terletak di akar pohon. Tidak ada input untuk Root Node, menandakan bahwa tidak ada yang akan memasuki node ini.
2. Internal Node adalah simpul percabangan pada node ini hanya ada satu input dan mungkin memiliki satu atau lebih output.
3. Leaf Node, node terakhir pohon keputusan dikenal sebagai node daun atau node terminal. Node saat ini hanya memiliki satu input dan tidak ada output.

Dalam penerapan klasifikasi, struktur pohon keputusan dibentuk melalui proses pembagian data berdasarkan atribut yang dinilai mampu membedakan kelas. Setiap keputusan pada simpul mengarahkan data menuju cabang tertentu hingga mencapai simpul daun. Oleh karena itu, hasil klasifikasi dapat ditelusuri kembali berdasarkan urutan aturan yang dilalui oleh data.

2.1.1 Algoritma Decision Tree CART

Algoritma Clasification and Regression Trees (CART) merupakan salah satu metode decision tree yang digunakan untuk melakukan proses klasifikasi dan regresi dengan membangun pohon keputusan biner. Proses penyusunan pohon keputusan pada algoritma CART dilakukan dengan menentukan nilai Gini Index pada setiap kelas sebagai dasar pemisahan data. Bentuk persamaan algoritma CART dinyatakan seperti yang diuraikan berikut:

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2$$

Nilai P_i diperoleh dengan membandingkan jumlah data pada atribut kelas C_i terhadap keseluruhan jumlah data. Proses pemisahan data menjadi dua subset, yaitu D1 dan D2, dilakukan berdasarkan nilai Gini Index terendah yang dapat dirumuskan sebagai berikut:

$$Gini_a = (|D_1| / |D|)\ Gini(D_1) + (|D_2| / |D|)\ Gini(D_2)$$

Nilai Gini Index digunakan untuk menggambarkan tingkat ketidakmurnian kelas dalam suatu kelompok data. Semakin kecil nilai Gini Index, semakin homogen komposisi kelas pada kelompok tersebut. Pada CART, pemilihan pemisahan dilakukan dengan membandingkan nilai ketidakmurnian tertimbang dari kandidat pembagian, kemudian memilih pembagian yang menghasilkan nilai paling rendah.

2.2 Naive Bayes

Pengklasifikasian Naive bayes adalah salah satu algoritma lain dari metode klasifikasi selain Decision Tree dan K-Nearest Neighbor (KNN). Naive Bayes bekerja dengan melakukan pengklasifikasian yang bersumber dari perhitungan probabilitas sederhana. Algoritma machine learning yang satu ini dirancang untuk dapat difungsikan berdasarkan asumsi setiap variabel yang independen. Secara umum, Naive Bayes biasanya digunakan untuk menemukan probabilitas lain dari data probabilitas tertentu yang telah dimiliki.

Secara statistik, metode yang dikenal sebagai Naive Bayes dapat digunakan untuk memperkirakan kemungkinan terjadinya suatu kelas. Naive Bayes sering digunakan untuk mengkategorikan data kumpulan yang diperoleh melalui pelatihan intensif. Selain itu, untuk klasifikasi probabilistik yang kuat yang mengurangi probabilitas total dengan menggabungkan frekuensi dan menggabungkan nilai numerik dari kumpulan data yang tersedia. Naive Bayes adalah teknik yang menggunakan probabilitas sederhana yang didasarkan pada teori inferensi Bayesian dengan asumsi independensi yang kuat. Manfaat lain dari Naive Bayes adalah dapat digunakan untuk menganalisis variabel dalam situasi yang paling bermasalah.

Naïve Bayes adalah metode klasifikasi statistik yang menghitung kemungkinan kesamaan antara kasus lama dan baru berdasarkan kasus, Naïve Bayes mampu memprediksi kelas keanggotaan probabilistik dan memiliki akurasi tinggi dan kecepatan yang baik jika diterapkan pada database besar. Naïve Bayes adalah metode supervised learning yang berarti metode ini membutuhkan data awal berupa dataset latih selama fase pembelajaran.

Teori dasar Naïve Bayes adalah menghitung probabilitas-probabilitas dari kategori dalam data untuk mendapatkan perkiraan probabilitas dari kategori yang diberikan, persamaan umum yang digunakan untuk teorema naïve bayes adalah seperti pada Persamaan

$$P(Y|X) = P(X|Y)\ P(Y) / P(X) \quad (9)$$

X adalah kriteria suatu kasus berdasarkan masukan, Y adalah kelas solusi pola label, P(Y|X) adalah probabilitas kemunculan label kelas Y dengan kriteria masukan X, P(X|Y) adalah probabilitas kemunculan label kelas X dengan kriteria masukan Y, P(Y) adalah probabilitas label kelas Y, dan P(X) adalah probabilitas label kelas X.

Naive Bayes adalah statistik klasifikasi yang dapat digunakan untuk memperkirakan kemungkinan setiap kelas tertentu akan memiliki anggota. Saat digunakan untuk database dengan jumlah data yang besar. Bayesian Classification memiliki kecepatan dan akurasi yang kuat. Metode Naive Bayes yang tergolong teknik statistik, dapat memprediksi kemungkinan terjadinya kelas kehancuran. Nave Bayes sering digunakan untuk mengkategorikan data himpunan dari pelatihan intensif. Teorema Bayes sebagai berikut:

$$P(C|A) = P(A|C)\ P(C) / P(A)$$

Perhitungan Naive Bayes dilakukan dengan memanfaatkan probabilitas awal setiap kelas serta probabilitas kemunculan fitur pada kelas tersebut. Hasil probabilitas dari masing-masing kelas kemudian dibandingkan untuk menentukan kelas prediksi. Asumsi independensi bersyarat antarf fitur membantu menyederhanakan proses perhitungan, walaupun hubungan antarfitur dalam data nyata tidak selalu sepenuhnya independen.

Dalam penelitian klasifikasi, Decision Tree dan Naive Bayes dapat digunakan pada data yang sama untuk mengetahui perbedaan hasil prediksi kedua algoritma. Decision Tree menghasilkan aturan keputusan yang berbentuk percabangan, sedangkan Naive Bayes memberikan prediksi berdasarkan perhitungan probabilitas. Evaluasi keduanya dapat dilakukan menggunakan data pengujian dan ukuran kinerja klasifikasi yang sesuai dengan tujuan penelitian.

BAB III
HASIL

3.1 Decision Tree – Naive Bayes

Import Library

202431075_Nuruliah Rifky

```
[1]: #202431075_Nuruliah Rifky
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay,
                             classification_report)
```

Pada tahap awal, saya melakukan import library yang diperlukan untuk mendukung proses pengolahan data dan klasifikasi menggunakan algoritma Decision Tree dan Naive Bayes. Library NumPy digunakan untuk melakukan perhitungan numerik, sedangkan Pandas digunakan untuk membaca dan mengolah dataset. Selanjutnya, Matplotlib digunakan untuk menampilkan visualisasi data, sementara train_test_split digunakan untuk membagi dataset menjadi data latih dan data uji. ColumnTransformer dan OneHotEncoder digunakan untuk mengolah data kategorikal menjadi bentuk numerik, sedangkan Pipeline digunakan untuk menyusun tahapan pengolahan data dan pemodelan secara terstruktur. Kemudian, DecisionTreeClassifier digunakan untuk membangun model klasifikasi Decision Tree dan plot_tree digunakan untuk menampilkan struktur pohon keputusan. GaussianNB digunakan untuk membangun model klasifikasi Naive Bayes. Selain itu, saya juga mengimpor beberapa fungsi evaluasi seperti accuracy, precision, recall, F1-score, ROC AUC, confusion matrix, dan classification report untuk mengukur serta membandingkan kinerja kedua algoritma dalam melakukan klasifikasi data.

Pre Processing

Membaca Dataset

202431075_Nuruliah Rifky

```
[2]: #202431075_Nuruliah Rifky
df = pd.read_csv('04_bank_customers.csv', sep=';')
df
```

[2]:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	no
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	no
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	no
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	no
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	no
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45206	51	technician	married	tertiary	no	825	no	no	cellular	17	nov	977	3	-1	0	unknown	yes
45207	71	retired	divorced	primary	no	1729	no	no	cellular	17	nov	456	2	-1	0	unknown	yes
45208	72	retired	married	secondary	no	5715	no	no	cellular	17	nov	1127	5	184	3	success	yes
45209	57	blue-collar	married	secondary	no	668	no	no	telephone	17	nov	508	4	-1	0	unknown	no
45210	37	entrepreneur	married	secondary	no	2971	no	no	cellular	17	nov	361	2	188	11	other	no

45211 rows × 17 columns

Pada tahap pre-processing, saya melakukan proses pembacaan dataset menggunakan library Pandas dengan perintah pd.read_csv('04_bank_customers.csv', sep=';'). Perintah tersebut digunakan untuk membaca file dataset dengan format CSV yang menggunakan

tanda titik koma (;) sebagai pemisah antar kolom. Selanjutnya, dataset disimpan ke dalam variabel df dan ditampilkan untuk melihat isi data yang akan diolah. Berdasarkan hasil yang ditampilkan, dataset memiliki 45.211 baris dan 17 kolom yang berisi informasi mengenai nasabah bank, seperti usia, pekerjaan, status pernikahan, pendidikan, saldo rekening, pinjaman, dan informasi lainnya. Dataset ini juga memiliki kolom y sebagai variabel target yang menunjukkan apakah nasabah berlangganan deposito berjangka atau tidak. Tahap ini saya lakukan untuk mengetahui struktur dan isi dataset sebelum melanjutkan proses pengolahan data serta penerapan algoritma Decision Tree dan Naive Bayes.

```
[3]: #202431075_Nuruliah Rifky
     df.info()
```

202431075_Nuruliah Rifky

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

Selanjutnya, dilakukan pengecekan informasi dataset menggunakan perintah df.info() untuk mengetahui jumlah baris, kolom, tipe data, serta kelengkapan data. Berdasarkan hasil yang ditampilkan, dataset memiliki 45.211 baris dan 17 kolom, yang terdiri dari 7 kolom bertipe int64 dan 10 kolom bertipe object. Seluruh kolom memiliki 45.211 data non-null, yang menunjukkan bahwa tidak terdapat data kosong pada dataset. Selain itu, penggunaan memori dataset sebesar 5,9 MB. Pengecekan ini bertujuan untuk mengetahui kondisi dan struktur data sebelum melanjutkan ke proses pengolahan data menggunakan algoritma Decision Tree dan Naive Bayes.

Ukuran dataset dan statistik deskriptif

```
[4]: #202431075_Nuruliah Rifky
     print('Ukuran dataset:', df.shape)
     display(df.describe(include='all'))
```

202431075_Nuruliah Rifky

```
Ukuran dataset: (45211, 17)
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
count	45211.000000	45211	45211	45211	45211	45211.000000	45211	45211	45211	45211.000000	45211	45211.000000	45211.000000	45211.000000	45211.000000	45211	45211
unique	NaN	12	3	4	2	NaN	2	2	3	NaN	12	NaN	NaN	NaN	NaN	4	2
top	NaN	blue-collar	married	secondary	no	NaN	yes	no	cellular	NaN	may	NaN	NaN	NaN	NaN	unknown	no
freq	NaN	9732	27214	23202	44396	NaN	25130	37967	29285	NaN	13766	NaN	NaN	NaN	NaN	36959	39922
mean	40.936210	NaN	NaN	NaN	NaN	1362.272058	NaN	NaN	NaN	15.806419	NaN	258.163080	2.763841	40.197828	0.580323	NaN	NaN
std	10.618762	NaN	NaN	NaN	NaN	3044.765829	NaN	NaN	NaN	8.322476	NaN	257.527812	3.098021	100.128746	2.303441	NaN	NaN
min	18.000000	NaN	NaN	NaN	NaN	-8019.000000	NaN	NaN	NaN	1.000000	NaN	0.000000	1.000000	-1.000000	0.000000	NaN	NaN
25%	33.000000	NaN	NaN	NaN	NaN	72.000000	NaN	NaN	NaN	8.000000	NaN	103.000000	1.000000	-1.000000	0.000000	NaN	NaN
50%	39.000000	NaN	NaN	NaN	NaN	448.000000	NaN	NaN	NaN	16.000000	NaN	180.000000	2.000000	-1.000000	0.000000	NaN	NaN
75%	48.000000	NaN	NaN	NaN	NaN	1428.000000	NaN	NaN	NaN	21.000000	NaN	319.000000	3.000000	-1.000000	0.000000	NaN	NaN
max	95.000000	NaN	NaN	NaN	NaN	102127.000000	NaN	NaN	NaN	31.000000	NaN	4918.000000	63.000000	871.000000	275.000000	NaN	NaN

Pada bagian ukuran dataset dan statistik deskriptif, saya menggunakan perintah df.shape untuk mengetahui jumlah baris dan kolom, sedangkan df.describe(include='all')

digunakan untuk melihat informasi statistik dari keseluruhan data. Hasil yang diperoleh menunjukkan bahwa dataset memiliki 45.211 baris dan 17 kolom. Dari hasil statistik deskriptif, terlihat bahwa rata-rata usia nasabah adalah 40,94 tahun, dengan usia terendah 18 tahun dan tertinggi 95 tahun. Selain itu, rata-rata saldo nasabah sebesar 1.362,27, dengan saldo terendah -8.019 dan tertinggi 102.127. Untuk data kategorikal, pekerjaan yang paling banyak adalah blue-collar, status pernikahan yang paling sering muncul adalah married, dan tingkat pendidikan yang paling umum adalah secondary. Pada kolom y sebagai target klasifikasi, kategori no merupakan nilai yang paling banyak muncul, yaitu sebanyak 39.922 data. Melalui hasil ini, dapat diketahui gambaran umum mengenai karakteristik nasabah dan persebaran data yang akan digunakan dalam proses klasifikasi menggunakan algoritma Decision Tree dan Naive Bayes.

Memeriksa missing value dan duplikasi

```
[5]: #202431075_Nuruliah Rifky
print('Missing value tiap kolom:')
print(df.isnull().sum())
print('Jumlah data duplikat:', df.duplicated().sum())
```

```
Missing value tiap kolom:
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64
Jumlah data duplikat: 0
```

202431075_Nuruliah Rifky

Setelah mengetahui ukuran dan statistik deskriptif dataset, tahap berikutnya adalah memeriksa missing value dan duplikasi data untuk memastikan tidak terdapat data kosong maupun data yang berulang. Pada pemeriksaan ini, saya menggunakan perintah df.isnull().sum() untuk mengetahui jumlah nilai kosong pada setiap kolom, sedangkan df.duplicated().sum() digunakan untuk memeriksa jumlah data yang duplikat. Berdasarkan hasil yang ditampilkan, seluruh kolom memiliki nilai missing value sebesar 0, yang menunjukkan bahwa tidak terdapat data kosong. Selain itu, jumlah data duplikat juga menunjukkan angka 0, sehingga tidak ditemukan data yang berulang. Dari hasil tersebut, dataset tidak memerlukan penghapusan data kosong maupun data duplikat dan dapat dilanjutkan ke proses pengolahan data berikutnya.

EDA (Exploratory Data Analysis)

Mengamati target dan beberapa variabel

```
[6]: #202431075_Nuruliah Rifky
print('Nilai unik target:')
print(df['y'].unique())
print('Distribusi target:')
print(df['y'].value_counts())
print('Persentase target:')
print((df['y'].value_counts(normalize=True)*100).round(2))
```

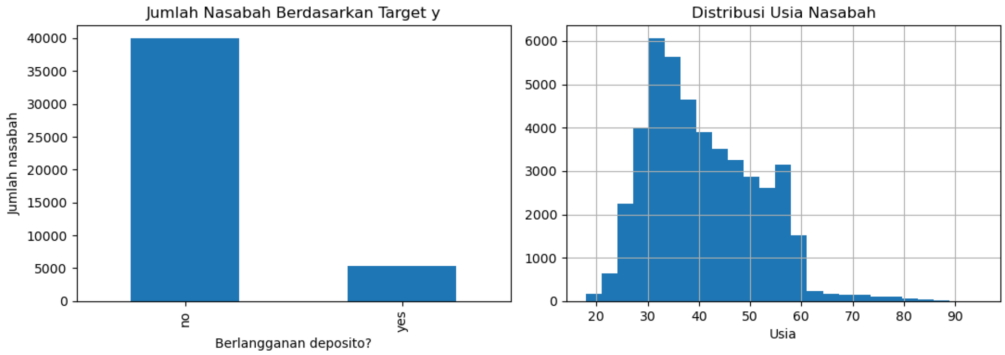
```
Nilai unik target:
['no' 'yes']
Distribusi target:
y
no     39922
yes     5289
Name: count, dtype: int64
Persentase target:
y
no     88.3
yes    11.7
Name: proportion, dtype: float64
```

202431075_Nuruliah Rifky

Setelah melakukan pemeriksaan missing value dan duplikasi data, proses dilanjutkan ke tahap EDA (Exploratory Data Analysis) untuk mengamati variabel target pada dataset. Pada bagian ini, saya menggunakan perintah df['y'].unique() untuk mengetahui nilai unik pada kolom target, df['y'].value_counts() untuk menghitung jumlah data pada setiap kategori, serta df['y'].value_counts(normalize=True) untuk melihat persentase masing-masing kategori. Berdasarkan hasil yang ditampilkan, variabel target memiliki dua kategori, yaitu no dan yes. Kategori no berjumlah 39.922 data dengan persentase 88,3%, sedangkan kategori yes berjumlah 5.289 data dengan persentase 11,7%. Dari hasil tersebut, terlihat bahwa jumlah data pada kategori no lebih banyak dibandingkan kategori yes, sehingga distribusi target tidak seimbang. Hasil pengamatan ini digunakan untuk memahami persebaran kelas target sebelum melanjutkan proses klasifikasi menggunakan algoritma Decision Tree dan Naive Bayes.

```
[7]: #202431075_Nuruliah Rifky
fig, ax = plt.subplots(1, 2, figsize=(11,4))
df['y'].value_counts().plot(kind='bar', ax=ax[0])
ax[0].set_title('Jumlah Nasabah Berdasarkan Target y')
ax[0].set_xlabel('Berlangganan deposito?')
ax[0].set_ylabel('Jumlah nasabah')
df['age'].hist(bins=25, ax=ax[1])
ax[1].set_title('Distribusi Usia Nasabah')
ax[1].set_xlabel('Usia')
plt.tight_layout()
plt.show()
```

202431075_Nuruliah Rifky



Pada bagian ini, saya membuat visualisasi data untuk melihat perbandingan jumlah nasabah berdasarkan target dan distribusi usia nasabah menggunakan library Matplotlib. Perintah plt.subplots(1, 2, figsize=(11,4)) digunakan untuk menampilkan dua grafik dalam satu tampilan agar lebih mudah dibandingkan. Grafik pertama menggunakan

diagram batang (bar chart) untuk menunjukkan jumlah nasabah berdasarkan kategori target y, sedangkan grafik kedua menggunakan histogram untuk melihat persebaran usia nasabah. Berdasarkan grafik pertama, terlihat bahwa jumlah nasabah dengan kategori no lebih banyak dibandingkan kategori yes, yaitu sebanyak 39.922 nasabah tidak berlangganan deposito dan 5.289 nasabah berlangganan deposito. Sementara itu, pada grafik kedua dapat dilihat bahwa sebagian besar nasabah berada pada rentang usia sekitar 30–40 tahun, sedangkan jumlah nasabah pada usia yang lebih tua cenderung lebih sedikit. Dari kedua grafik tersebut, saya dapat melihat perbedaan jumlah nasabah berdasarkan target serta mengetahui pola persebaran usia nasabah.

Encoding variabel target (seperti LabelEncoder pada contoh Iris)

[8]:
```
#202431075_Nuruliah Rifky
df['y'] = df['y'].map({'no':0, 'yes':1})
print(df['y'].unique())
df.head()
```

202431075_Nuruliah Rifky

```
[0 1]
```

[8]:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	0
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	0
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	0
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	0
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	0

Pada tahap ini, saya melakukan proses encoding pada variabel target y dengan mengubah data kategorikal menjadi data numerik. Proses ini dilakukan menggunakan fungsi map() untuk mengubah nilai 'no' menjadi 0 dan 'yes' menjadi 1. Kemudian, fungsi unique() digunakan untuk melihat nilai unik pada variabel target setelah proses encoding, yaitu [0 1]. Setelah itu, fungsi df.head() digunakan untuk menampilkan lima baris pertama dataset. Berdasarkan hasil yang ditampilkan, dapat dilihat bahwa variabel target y sudah berhasil diubah menjadi bentuk numerik sehingga dapat digunakan dalam proses pemodelan machine learning.

Encoding fitur kategorikal 202431075_Nuruliah Rifky

[9]:
```
#202431075_Nuruliah Rifky
print('Kolom kategorikal:')
print(df.select_dtypes(include='object').columns.tolist())
print('Contoh isi kolom job:', df['job'].unique())
```

```
Kolom kategorikal:
['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']
Contoh isi kolom job: ['management' 'technician' 'entrepreneur' 'blue-collar' 'unknown'
 'retired' 'admin.' 'services' 'self-employed' 'unemployed' 'housemaid'
 'student']
```

Selanjutnya, saya melakukan pengecekan terhadap fitur kategorikal yang terdapat dalam dataset sebelum dilakukan proses encoding. Pada tahap ini, kode df.select_dtypes(include='object').columns.tolist() digunakan untuk menampilkan kolom yang memiliki tipe data kategorikal. Berdasarkan hasil yang diperoleh, terdapat 9 kolom kategorikal, yaitu job, marital, education, default, housing, loan, contact, month, dan poutcome. Kemudian, kode df['job'].unique() digunakan untuk mengetahui berbagai jenis pekerjaan yang terdapat pada kolom job, seperti management, technician, entrepreneur, blue-collar, dan jenis pekerjaan lainnya.

Modeling

Menentukan fitur X dan target y

```
[10]: #202431075_Nuruliah Rifky
x = df.drop(['y'], axis=1)
y = df['y']
print('Fitur x:')
display(x.head())
print('Target y:')
print(y.head())
```

202431075_Nuruliah Rifky

Fitur x:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown

```
Target y:
0    0
1    0
2    0
3    0
4    0
Name: y, dtype: int64
```

Pada tahap modeling, saya menentukan fitur X dan target y yang akan digunakan dalam proses pemodelan machine learning. Kode x = df.drop(['y'], axis=1) digunakan untuk mengambil seluruh kolom pada dataset kecuali kolom y, sehingga kolom lainnya dijadikan sebagai fitur X. Sedangkan kode y = df['y'] digunakan untuk menentukan kolom y sebagai variabel target yang akan diprediksi. Kemudian, fungsi display(x.head()) digunakan untuk menampilkan lima baris pertama dari fitur X, sedangkan print(y.head()) digunakan untuk menampilkan lima nilai pertama dari target y. Berdasarkan hasil yang ditampilkan, terdapat 16 fitur X yang digunakan sebagai variabel input, sedangkan target y berisi nilai 0 dan 1 yang menunjukkan hasil encoding sebelumnya.

Membagi data training dan testing

```
[11]: #202431075_Nuruliah Rifky
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.20, random_state=42, stratify=y)
print('Jumlah train:', len(x_train))
print('Jumlah test:', len(x_test))
```

202431075_Nuruliah Rifky

```
Jumlah train: 36168
Jumlah test: 9043
```

Selanjutnya, saya melakukan pembagian dataset menjadi data training dan data testing menggunakan fungsi train_test_split(). Pada tahap ini, parameter test_size=0.20 digunakan untuk membagi data dengan perbandingan 80% sebagai data training dan 20% sebagai data testing. Kemudian, parameter random_state=42 digunakan agar hasil pembagian data tetap sama setiap kali kode dijalankan, sedangkan stratify=y digunakan untuk menjaga proporsi kelas pada variabel target agar tetap seimbang sesuai dengan distribusi data awal. Setelah itu, fungsi len(x_train) dan len(x_test) digunakan untuk mengetahui jumlah data pada masing-masing bagian. Berdasarkan hasil yang diperoleh, terdapat 36.168 data training dan 9.043 data testing.

Persiapan preprocessing fitur

```
[12]: #202431075_Nuruliah Rifky
kolom_numerik = x.select_dtypes(include='number').columns.tolist()
kolom_kategori = x.select_dtypes(include='object').columns.tolist()
try:
    encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    encoder = OneHotEncoder(handle_unknown='ignore', sparse=False)
preprocessing = ColumnTransformer([('num', 'passthrough', kolom_numerik), ('kat', encoder, kolom_kategori)])
print('Numerik:', kolom_numerik)
print('Kategorikal:', kolom_kategori)
```

202431075_Nuruliah Rifky

```
Numerik: ['age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous']
Kategorikal: ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']
```

Pada tahap ini, saya melakukan persiapan preprocessing fitur dengan memisahkan kolom numerik dan kolom kategorikal yang terdapat dalam dataset. Fungsi select_dtypes() digunakan untuk mengidentifikasi kolom berdasarkan tipe datanya, sedangkan columns.tolist() digunakan untuk menampilkan nama kolom dalam bentuk daftar. Kemudian, saya menggunakan OneHotEncoder untuk mengubah data kategorikal menjadi bentuk numerik agar dapat diproses oleh model machine learning. Parameter handle_unknown='ignore' digunakan agar kategori baru yang tidak ditemukan saat pelatihan tidak menyebabkan error, sedangkan sparse_output=False digunakan untuk menghasilkan output dalam bentuk array biasa. Selain itu, digunakan try dan except TypeError untuk menyesuaikan perbedaan versi library. Setelah itu, ColumnTransformer digunakan untuk menggabungkan proses preprocessing, yaitu mempertahankan kolom numerik tanpa perubahan dan melakukan encoding pada kolom kategorikal. Berdasarkan hasil yang ditampilkan, terdapat 7 kolom numerik dan 9 kolom kategorikal.

Decision Tree

Membuat dan melatih model

```
[13]: #202431075_Nuruliah Rifky
classifier = Pipeline([
    ('preprocessing', preprocessing),
    ('tree', DecisionTreeClassifier(
        criterion='gini',
        max_depth=6,
        min_samples_leaf=30,
        random_state=42
    ))
])

classifier.fit(x_train, y_train)
print('Decision Tree berhasil dilatih')
```

202431075_Nuruliah Rifky

```
Decision Tree berhasil dilatih
```

Selanjutnya, saya membuat dan melatih model Decision Tree menggunakan Pipeline yang menggabungkan proses preprocessing dengan algoritma DecisionTreeClassifier. Dalam pembuatan model ini, parameter criterion='gini' digunakan untuk menentukan pemisahan data berdasarkan tingkat ketidakmurnian kelas. Parameter max_depth=6 digunakan untuk membatasi kedalaman pohon keputusan agar model tidak terlalu kompleks, sedangkan min_samples_leaf=30 digunakan untuk menentukan jumlah minimum 30 sampel pada setiap daun pohon keputusan. Selain itu, parameter random_state=42 digunakan agar hasil pelatihan dapat diperoleh secara konsisten ketika kode dijalankan kembali. Kemudian, saya menggunakan classifier.fit(x_train, y_train) untuk melatih model menggunakan data training yang telah disiapkan sebelumnya.

Berdasarkan hasil yang ditampilkan, muncul keterangan "Decision Tree berhasil dilatih" yang menunjukkan bahwa proses pelatihan model telah berhasil dijalankan tanpa mengalami error.

Visualisasi pohon keputusan

```
[14]: #202431075_Nuruliah Rifky

from sklearn import tree

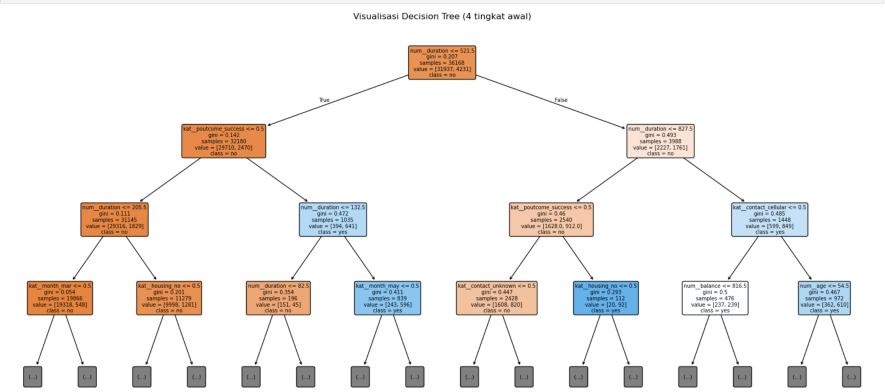
nama_fitur = classifier.named_steps[
    'preprocessing'
].get_feature_names_out()

plt.figure(figsize=(17, 8))

tree.plot_tree(
    classifier.named_steps['tree'],
    feature_names=nama_fitur,
    class_names=['no', 'yes'],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=7
)

plt.title(
    'Visualisasi Decision Tree (4 tingkat awal)'
)
plt.tight_layout()
plt.show()
```

202431075_Nuruliah Rifky



Selanjutnya, saya melakukan visualisasi pohon keputusan dari model Decision Tree yang telah dilatih sebelumnya. Visualisasi ini dibuat menggunakan fungsi tree.plot_tree() untuk menampilkan struktur pohon keputusan beserta aturan pemisahan data pada setiap percabangannya. Kode classifier.named_steps['preprocessing'].get_feature_names_out() digunakan untuk mengambil nama fitur setelah proses preprocessing, sedangkan classifier.named_steps['tree'] digunakan untuk mengambil model Decision Tree yang telah dilatih. Parameter feature_names digunakan untuk menampilkan nama fitur, sedangkan class_names=['no', 'yes'] digunakan untuk memberikan nama pada kelas target. Selain itu, parameter filled=True dan rounded=True digunakan untuk memberikan warna serta membuat bentuk kotak lebih rapi, sedangkan max_depth=3 digunakan untuk membatasi visualisasi hingga empat tingkat awal agar lebih mudah dibaca. Berdasarkan hasil visualisasi, terlihat bahwa pohon keputusan memiliki beberapa percabangan yang menunjukkan proses pemisahan data berdasarkan kondisi fitur tertentu. Setiap kotak menampilkan informasi seperti nilai gini, jumlah sampel, distribusi kelas, dan hasil

klasifikasi. Warna oranye menunjukkan dominasi kelas no, sedangkan warna biru menunjukkan dominasi kelas yes. Visualisasi ini bertujuan untuk mempermudah pemahaman mengenai cara kerja model Decision Tree dalam menentukan hasil klasifikasi berdasarkan fitur yang digunakan.

Kedalaman pohon dan akurasi

```
[15]: #202431075_Nuruliah Rifky
depth = classifier.named_steps['tree'].get_depth()
print('Kedalaman pohon:', depth)
y_pred_tree = classifier.predict(x_test)
akurasi_tree = accuracy_score(y_test, y_pred_tree)
print('Akurasi Decision Tree:', akurasi_tree)
print('Akurasi dalam persen: {:.2f}%'.format(akurasi_tree*100))
```

202431075_Nuruliah Rifky

```
Kedalaman pohon: 6
Akurasi Decision Tree: 0.9009178370009953
Akurasi dalam persen: 90.09%
```

Selanjutnya, saya melakukan pengecekan kedalaman pohon dan menghitung tingkat akurasi model Decision Tree yang telah dilatih sebelumnya. Fungsi get_depth() digunakan untuk mengetahui kedalaman pohon keputusan yang terbentuk, sedangkan classifier.predict(x_test) digunakan untuk melakukan prediksi terhadap data testing. Kemudian, fungsi accuracy_score(y_test, y_pred_tree) digunakan untuk menghitung tingkat akurasi dengan membandingkan hasil prediksi model dengan nilai target sebenarnya. Hasil akurasi tersebut dikalikan 100 untuk mengubahnya menjadi bentuk persentase. Berdasarkan hasil yang diperoleh, model Decision Tree memiliki kedalaman pohon sebesar 6 dengan nilai akurasi 0,900917837 atau sebesar 90,09%. Hal ini menunjukkan bahwa model mampu melakukan prediksi dengan benar pada sekitar 90,09% data testing, sedangkan sekitar 9,91% lainnya belum diprediksi dengan tepat. Dengan demikian, model Decision Tree memiliki tingkat akurasi yang cukup tinggi dalam melakukan klasifikasi terhadap dataset yang digunakan.

Naive Bayes

Membuat dan melatih model Naive Bayes

```
[16]: #202431075_Nuruliah Rifky
naive_bayes = Pipeline([('preprocessing', preprocessing), ('nb', GaussianNB())])
naive_bayes.fit(x_train, y_train)
y_pred_nb = naive_bayes.predict(x_test)
akurasi_nb = accuracy_score(y_test, y_pred_nb)
print('Akurasi Naive Bayes:', akurasi_nb)
print('Akurasi dalam persen: {:.2f}%'.format(akurasi_nb*100))
```

202431075_Nuruliah Rifky

```
Akurasi Naive Bayes: 0.8564635629768882
Akurasi dalam persen: 85.65%
```

Pada bagian ini, saya membuat dan melatih model Naive Bayes menggunakan Pipeline yang menggabungkan proses preprocessing dengan algoritma GaussianNB. Penggunaan Pipeline bertujuan agar proses pengolahan data dan pelatihan model dapat dilakukan secara berurutan. Fungsi naive_bayes.fit(x_train, y_train) digunakan untuk melatih model menggunakan data training yang telah disiapkan sebelumnya, sedangkan naive_bayes.predict(x_test) digunakan untuk melakukan prediksi terhadap data testing. Kemudian, fungsi accuracy_score(y_test, y_pred_nb) digunakan untuk menghitung tingkat akurasi dengan membandingkan hasil prediksi model dengan nilai target

sebenarnya. Hasil akurasi tersebut dikalikan 100 agar dapat ditampilkan dalam bentuk persentase. Berdasarkan hasil yang diperoleh, model Naive Bayes memiliki nilai akurasi sebesar 0,856463563 atau 85,65%. Hal ini menunjukkan bahwa model mampu memprediksi dengan benar sekitar 85,65% data testing, sedangkan sekitar 14,35% lainnya belum diprediksi dengan tepat. Dengan demikian, model Naive Bayes memiliki kemampuan yang cukup baik dalam melakukan klasifikasi terhadap dataset yang digunakan.

Hasil evaluasi kedua model

[17]:
```
#202431075_Nuruliah Rifky

model = {
    'Decision Tree': classifier,
    'Naive Bayes': naive_bayes
}
hasil = []

for nama, algoritma in model.items():
    prediksi = algoritma.predict(x_test)
    peluang = algoritma.predict_proba(x_test)[:, 1]

    hasil.append({
        'Algoritma': nama,
        'Accuracy': accuracy_score(y_test, prediksi),
        'Precision': precision_score(y_test, prediksi, zero_division=0),
        'Recall': recall_score(y_test, prediksi, zero_division=0),
        'F1-score': f1_score(y_test, prediksi, zero_division=0),
        'ROC-AUC': roc_auc_score(y_test, peluang)
    })

    print('\nClassification Report:', nama)
    print(classification_report(
        y_test, prediksi, target_names=['no', 'yes']
    ))

perbandingan = pd.DataFrame(hasil).set_index('Algoritma')
display(perbandingan.round(4))
```

202431075_Nuruliah Rifky

```
Classification Report: Decision Tree
              precision    recall  f1-score   support

          no       0.93      0.97      0.95      7985
         yes       0.61      0.41      0.49      1058

    accuracy                           0.90      9043
   macro avg       0.77      0.69      0.72      9043
weighted avg       0.89      0.90      0.89      9043
```

202431075_Nuruliah Rifky

```
Classification Report: Naive Bayes
              precision    recall  f1-score   support

          no       0.94      0.90      0.92      7985
         yes       0.42      0.56      0.48      1058

    accuracy                           0.86      9043
   macro avg       0.68      0.73      0.70      9043
weighted avg       0.88      0.86      0.87      9043
```

Algoritma	Accuracy	Precision	Recall	F1-score	ROC-AUC
Decision Tree	0.9009	0.6134	0.4140	0.4944	0.8654
Naive Bayes	0.8565	0.4156	0.5586	0.4766	0.8210

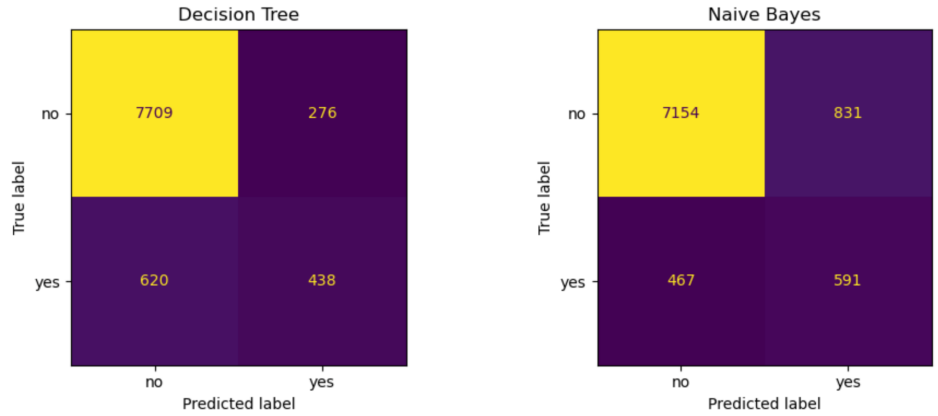
Pada bagian hasil evaluasi kedua model, saya melakukan perbandingan kinerja algoritma Decision Tree dan Naive Bayes menggunakan beberapa metrik evaluasi, yaitu accuracy, precision, recall, F1-score, dan ROC-AUC. Kode model.items() digunakan untuk mengambil kedua algoritma yang telah dilatih, sedangkan predict() digunakan untuk menghasilkan prediksi terhadap data testing dan predict_proba() digunakan untuk

memperoleh probabilitas prediksi pada kelas positif. Kemudian, hasil evaluasi dari masing-masing algoritma disimpan menggunakan hasil.append() dan ditampilkan melalui classification_report() untuk mengetahui nilai precision, recall, F1-score, serta jumlah data pada setiap kelas. Setelah itu, pd.DataFrame(hasil) digunakan untuk menyusun hasil evaluasi ke dalam bentuk tabel agar lebih mudah dibandingkan. Berdasarkan hasil yang diperoleh, Decision Tree memiliki nilai accuracy sebesar 90,09%, precision 61,34%, recall 41,40%, F1-score 49,44%, dan ROC-AUC 86,54%. Sementara itu, Naive Bayes memperoleh nilai accuracy sebesar 85,65%, precision 41,56%, recall 55,86%, F1-score 47,66%, dan ROC-AUC 82,10%. Hasil tersebut menunjukkan bahwa Decision Tree memiliki kinerja lebih tinggi pada accuracy, precision, F1-score, dan ROC-AUC, sedangkan Naive Bayes memiliki nilai recall yang lebih tinggi. Dengan demikian, Decision Tree menunjukkan kinerja yang lebih baik berdasarkan sebagian besar metrik evaluasi, sementara Naive Bayes lebih mampu mengidentifikasi data yang termasuk dalam kelas positif (yes).

Confusion Matrix

```
[18]: #202431075_Nuruliah Rifky
fig, axes = plt.subplots(1,2,figsize=(10,4))
for ax, (nama, algoritma) in zip(axes, model.items()):
    cm = confusion_matrix(y_test, algoritma.predict(x_test))
    ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['no','yes']).plot(ax=ax, colorbar=False)
    ax.set_title(nama)
plt.tight_layout()
plt.show()
```

202431075_Nuruliah Rifky



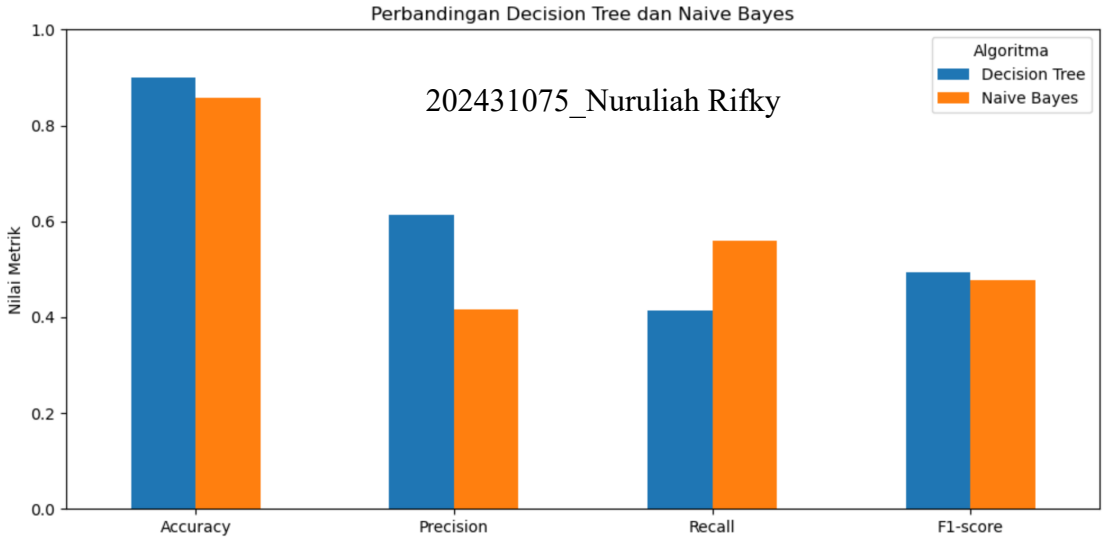
Untuk mengetahui hasil prediksi dari model Decision Tree dan Naive Bayes, saya melakukan visualisasi Confusion Matrix menggunakan fungsi confusion_matrix() untuk membandingkan nilai target sebenarnya dengan hasil prediksi masing-masing model. Fungsi ConfusionMatrixDisplay() digunakan untuk menampilkan hasil perbandingan dalam bentuk matriks, sedangkan plt.subplots(1,2) digunakan untuk menampilkan kedua Confusion Matrix secara berdampingan agar lebih mudah dibandingkan. Berdasarkan hasil yang diperoleh, model Decision Tree berhasil memprediksi 7.709 data kelas no dan 438 data kelas yes dengan benar, sedangkan 276 data kelas no salah diprediksi sebagai yes dan 620 data kelas yes salah diprediksi sebagai no. Sementara itu, model Naive Bayes berhasil memprediksi 7.154 data kelas no dan 591 data kelas yes dengan benar, sedangkan 831 data kelas no salah diprediksi sebagai yes dan 467 data kelas yes salah diprediksi sebagai no. Hasil tersebut menunjukkan bahwa Decision Tree memiliki jumlah prediksi benar yang lebih banyak secara keseluruhan, sedangkan Naive Bayes lebih baik dalam mengenali kelas positif (yes). Dengan demikian, Confusion Matrix membantu saya

mengetahui perbedaan kemampuan kedua algoritma dalam melakukan klasifikasi serta melihat jumlah prediksi yang benar dan salah pada masing-masing kelas.

Grafik perbandingan akurasi

```
[19]: #202431075_Nuruliah Rifky
perbandingan[['Accuracy','Precision','Recall','F1-score']].T.plot(kind='bar', figsize=(10,5), ylim=(0,1))
plt.title('Perbandingan Decision Tree dan Naive Bayes')
plt.ylabel('Nilai Metrik')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
perbandingan.to_csv('hasil_evaluasi_sesuai_contoh.csv')
```

202431075_Nuruliah Rifky



Untuk mempermudah perbandingan kinerja kedua algoritma, saya membuat grafik batang menggunakan data hasil evaluasi Decision Tree dan Naive Bayes. Fungsi plot(kind='bar') digunakan untuk menampilkan perbandingan nilai accuracy, precision, recall, dan F1-score dalam bentuk grafik batang. Kemudian, parameter figsize=(10,5) digunakan untuk mengatur ukuran grafik, sedangkan ylim=(0,1) digunakan untuk menentukan rentang nilai pada sumbu vertikal. Fungsi plt.title() digunakan untuk memberikan judul grafik, plt.ylabel() untuk memberikan keterangan sumbu vertikal, dan plt.xticks(rotation=0) untuk mengatur posisi label agar mudah dibaca. Selain itu, plt.tight_layout() digunakan untuk merapikan tampilan grafik, sedangkan perbandingan.to_csv() digunakan untuk menyimpan hasil evaluasi ke dalam file CSV. Berdasarkan grafik yang ditampilkan, Decision Tree memperoleh nilai accuracy sebesar 90,09%, precision 61,34%, dan F1-score 49,44%, lebih tinggi dibandingkan Naive Bayes yang memperoleh accuracy 85,65%, precision 41,56%, dan F1-score 47,66%. Namun, Naive Bayes memiliki nilai recall sebesar 55,86%, lebih tinggi dibandingkan Decision Tree sebesar 41,40%. Hasil tersebut menunjukkan bahwa Decision Tree memiliki kinerja lebih baik berdasarkan sebagian besar metrik evaluasi, sedangkan Naive Bayes lebih baik dalam mengenali kelas positif. Dengan demikian, grafik ini membantu saya melihat perbedaan kinerja kedua algoritma secara lebih jelas dan menentukan model yang lebih sesuai berdasarkan hasil evaluasi yang diperoleh.

BAB IV

PENUTUP

Berdasarkan landasan teori dan hasil praktikum UTS Pembelajaran Mesin yang telah dilakukan, diperoleh beberapa kesimpulan sebagai berikut:

1. Algoritma Decision Tree dan Naive Bayes dapat diterapkan untuk mengklasifikasikan data nasabah bank.
2. Dataset yang digunakan terdiri dari 45.211 baris dan 17 kolom, tanpa ditemukan missing value maupun data duplikat.
3. Proses klasifikasi meliputi preprocessing, encoding, pembagian data training dan testing, serta pelatihan model.
4. Decision Tree melakukan klasifikasi berdasarkan pohon keputusan, sedangkan Naive Bayes menggunakan perhitungan probabilitas.
5. Hasil pengujian menunjukkan akurasi Decision Tree sebesar 90,09% dan Naive Bayes sebesar 85,65%.
6. Decision Tree lebih unggul pada accuracy, precision, F1-score, dan ROC-AUC, sedangkan Naive Bayes lebih unggul pada recall.
7. Secara keseluruhan, Decision Tree menunjukkan kinerja lebih baik berdasarkan sebagian besar metrik evaluasi pada dataset yang digunakan.

DAFTAR PUSTAKA

[1] Adhitya, R. R., Witanti, W., & Yuniarti, R. (2023). Perbandingan Metode Cart Dan Naïve Bayes Untuk Klasifikasi Customer Churn. INFOTECH journal, 9(2), 307-318. https://ejournal.unma.ac.id/index.php/infotech/id/article/download/5641/3215

[2] Aprilia, K., Khaira, U., & Hutabarat, B. F. (2025). Komparasi Algoritma Naive Bayes Dan Decision Tree Pada Klasifikasi Penerimaan Peserta Didik Baru. Simtek: jurnal sistem informasi dan teknik komputer, 10(2), 547-553. https://ejournal.catursakti.ac.id/index.php/simtek/article/download/1694/411

[3] Biyantoro, A. S., & Prasetyo, B. (2024). Penerapan Decision Tree untuk Klasifikasi Status Kesehatan dengan perbandingan KNN dan Naive Bayes: Application of Decision Tree for Health Status Classification, Compared to KNN and Naive Bayes. Indonesian Journal of Informatic Research and Software Engineering (IJIRSE), 4(1), 47-55. https://journal.irpi.or.id/index.php/ijirse/article/download/1342/669

[4] Mardiani, E., Rahmansyah, N., Ningsih, S., Lantana, D. A., Wirawan, A. S. P., Wijaya, S. A., & Putri, D. N. (2023). Komparasi metode knn, naive bayes, decision tree, ensemble, linear regression terhadap analisis performa pelajar sma. Innovative: Journal Of Social Science Research, 3(2), 13880-13892. https://j-innovative.org/index.php/Innovative/article/view/1949/1468

[5] Novianto, E., Hermawan, A., & Avianto, D. (2023). Klasifikasi algoritma k-nearest neighbor, naive bayes, decision tree untuk prediksi status kelulusan mahasiswa s1. Rabit: Jurnal Teknologi dan Sistem Informasi Univrab, 8(2), 146-154. https://d1wqtxts1xzle7.cloudfront.net/114232167/1538-libre.pdf?1715031970=&response-content-disposition=inline%3B+filename%3DKlasifikasi_Algoritma_K_Nearest_Neighbor.pdf&Expires=1794068528&Signature=AO3EgX5ODH1V4xOkH47bFvcKLQMxzHyJt-esZk0SJcHA86HwyB5e-ijGdLUvFoETJLnks~xmH1uE6~4UYVqI7bf4dePnlqX2J-Rs8jNhFItePP~v-ct~xPVQ1VBZD5oyFR86mPhcst7xMGaC-2cIrSBvDIfELr-JvvYwKKkR4EoMH6zSXcrowHrPK75PM0ZpLfXNUy1xJTjlNwNIXNoEkR2lJ-wxJjF9M9MJp6c6He0KHNhiUaauOFu8Jurf--FobImyQ3TjlSLYsl80yi2PJKOQQB8w-KegFykNXPymHuZIIpJLQP1477TNmO6eSu34bUx7JDsCp-yLnH6U3AJaBA__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA